

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Problem Solving Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

COURSE CODE: MLA01

CO2: *Analyze and select appropriate agent strategies for different types of environments and problem settings. (BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100 Time: 60 Mins No. of Questions: 5

Annexure A

Problem Solving Assignment

Code of Conduct:

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation Total	10% 100%	

Signature of the Faculty

Total Marks Awarded

Q1 — Smart Ambulance Navigation

Given

The ambulance must travel from **S (Hospital)** to **G (Emergency location)**.

A* uses:

$$f(n)=g(n)+h(n)$$

where:

- $g(n)$ = actual cost from S to node n
- $h(n)$ = estimated cost from n to G
- $f(n)$ = total estimated cost through n

The heuristic values are:

Node $h(n)$

S	8
A	7
B	6
C	2
D	6
E	3
F	1
G	0

1. Problem Analysis

- **Initial state:** S

- **Actions:** Move from one connected junction to another.
- **Path cost:** Sum of the road costs.
- **Heuristic:** Estimated remaining distance $h(n)$.
- **Algorithm:** A* Search.

The important road costs from the diagram are:

Road Cost

$S \rightarrow A$ 2

$S \rightarrow D$ 6

$A \rightarrow D$ 6

$A \rightarrow E$ 4

$A \rightarrow B$ 4

$D \rightarrow E$ 3

$E \rightarrow B$ 2

$B \rightarrow C$ 3

$B \rightarrow G$ 6

$C \rightarrow G$ 2

$E \rightarrow F$ 4

$F \rightarrow G$ 2

2. A* Search Calculation

Step 1: Start at S

For S:

$$g(S)=0 \quad h(S)=8$$

Therefore:

$$f(S)=0+8=8$$

Expand S.

Its successors are **A** and **D**.

For A:

$$g(A)=0+2=2 \quad h(A)=7 \quad f(A)=2+7=9$$

For D:

$$g(D)=0+6=6 \quad h(D)=6 \quad f(D)=6+6=12$$

Choose **A**, because it has the smallest f-value.

Step 2: Expand A

From A we can reach B, D and E.

For B:

$$g(B)=2+4=6 \quad h(B)=6 \quad f(B)=6+6=12$$

For E:

$$g(E)=2+4=6 \quad h(E)=3 \quad f(E)=6+3=9$$

For D:

$$g(D)=2+6=8$$

But we already have a cheaper route to D:

$$g(D)=6$$

So retain:

$$f(D)=6+6=12$$

The OPEN list now contains approximately:

Node	$g(n)$	$h(n)$	$f(n)$
------	--------	--------	--------

E	6	3	9
---	---	---	----------

B	6	6	12
---	---	---	----

D	6	6	12
---	---	---	----

Choose **E**.

Step 3: Expand E

From E we can reach D, B and F.

F:

$$g(F)=g(E)+4 \quad g(F)=6+4=10 \quad h(F)=1$$

Therefore:

$$f(F)=10+1=11$$

B:

$$g(B)=6+2=8$$

But B already has a cheaper path:

$$g(B)=6$$

So retain:

$$f(B)=6+6=12$$

D:

$$g(D)=6+3=9$$

Again, the previous value $g(D)=6$ is cheaper.

So:

$$f(D)=6+6=12$$

OPEN list:

Node g(n) h(n) f(n)

F 10 1 **11**

B 6 6 12

D 6 6 12

Choose **F**.

Step 4: Expand F

F is connected to G.

For G:

$$g(G)=g(F)+2 \quad g(G)=10+2=12$$

Since:

$$h(G)=0$$

we get:

$$f(G)=12+0=12$$

OPEN list:

Node g(n) h(n) f(n)

B 6 6 12

D 6 6 12

G 12 0 12

There is a tie between B, D and G. Following the usual queue/order of insertion, **B is expanded before G.**

Step 5: Expand B

From B we can reach C and G.

For C:

$$g(C)=g(B)+3 \quad g(C)=6+3=9 \quad h(C)=2$$

Therefore:

$$f(C)=9+2=11$$

For G through B:

$$g(G)=6+6=12 \quad f(G)=12+0=12$$

So C has the smaller f-value.

Choose **C**.

Step 6: Expand C

C is connected to G.

$$g(G)=g(C)+2 \quad g(G)=9+2 \quad g(G)=11$$

Since:

$$h(G)=0 \quad f(G)=11+0=11$$

Now G is selected as the goal.

Therefore, the search terminates.

3. A* Exploration Table

This is the important table you can write in your assignment:

Step Node Expanded g(n) h(n) f(n)=g+h

1	S	0	8	8
2	A	2	7	9
3	E	6	3	9
4	F	10	1	11
5	B	6	6	12
6	C	9	2	11
7	G	11	0	11

Thus, the sequence of expanded nodes is:

$S \rightarrow A \rightarrow E \rightarrow F \rightarrow B \rightarrow C \rightarrow G$

The important point is that **D is generated but never expanded**, because its $f=12$, while better nodes remain available.

4. Selected Optimal Route

Although A* explores several nodes, the final parent relationships give:

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

Calculate the total cost:

$S \rightarrow A = 2$ $A \rightarrow B = 4$ $B \rightarrow C = 3$ $C \rightarrow G = 2$

Therefore:

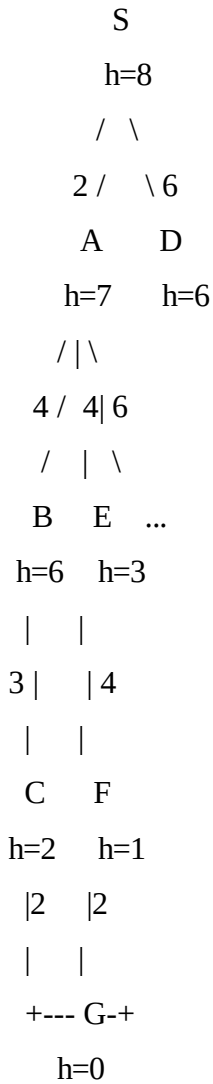
$2 + 4 + 3 + 2 = 11$

Final Answer

Optimal Route = $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$ Total Cost = 11

5. State-Space Representation

You can draw the search process like this:



The **final selected route** should be highlighted as:

S —2—> A —4—> B —3—> C —2—> G

Total cost = 2 + 4 + 3 + 2
= 11

6. Is the Route Optimal?

Yes, the selected route has total cost **11**.

We can compare some alternative routes:

Route 1

$$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G \quad 2+4+3+2=11$$

Route 2

$$S \rightarrow A \rightarrow E \rightarrow F \rightarrow G \quad 2+4+4+2=12$$

Route 3

$$S \rightarrow D \rightarrow E \rightarrow F \rightarrow G \quad 6+3+4+2=15$$

Route 4

$$S \rightarrow A \rightarrow B \rightarrow G \quad 2+4+6=12$$

Hence:

$$11 < 12 < 15$$

Therefore, $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$ is the shortest route among these alternatives.

Important note about the heuristic

There is a subtle point worth mentioning for a high-quality answer: the supplied heuristic has $h(B)=6$, while the actual cheapest remaining cost from B to G is $3+2=5$. So $h(B)$ is not strictly admissible at B.

Complexity Analysis

For A*:

- Let b = branching factor.
- Let d = depth of the optimal solution.

The worst-case time complexity is approximately:

$$O(b^d)$$

and space complexity is also approximately:

$$O(b^d)$$

because A* stores generated nodes in its OPEN/CLOSED structures.

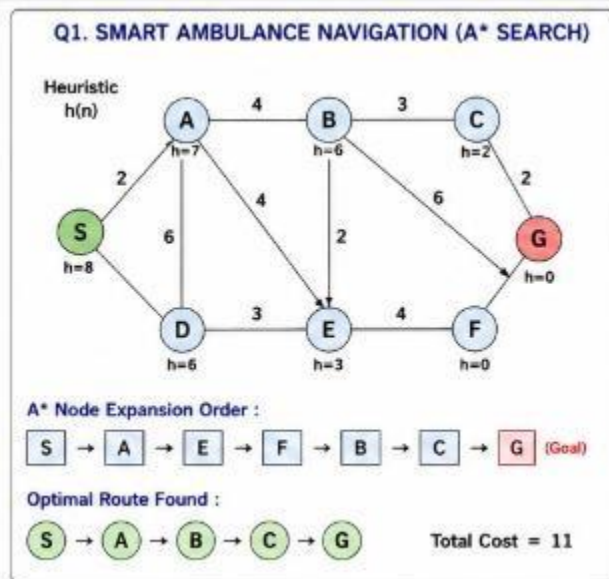
A* is more efficient than uninformed search when the heuristic provides useful guidance toward the goal because it considers both:

$$g(n)$$

and

$$h(n)$$

rather than only the path cost.



Final Assignment Conclusion

A* Search was applied to the smart ambulance road network using $f(n)=g(n)+h(n)$. The nodes were explored according to their evaluation values, and the final route obtained was $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$. The total travel cost is **11**, which is lower than the other feasible routes considered. Therefore, this route provides the best route for the ambulance in the given network.

Q2 — Autonomous Warehouse Robot

1. Problem Analysis

The robot must move from the **Charging Station S** to the **Goal G**. The important difficulty is that the robot **does not initially know the locations of all obstacles**. It discovers obstacles while moving.

Therefore, a normal offline A* search using a completely known map is not directly applicable.

Suitable strategy: Online A Search with Replanning*

The robot:

1. Starts with an unknown map.
2. Uses the **Manhattan distance** as the heuristic.
3. Moves toward the cell with the smallest estimated cost.
4. Senses nearby cells.
5. When an obstacle is discovered, it marks that cell as blocked.
6. Recalculates the best path using the updated map.
7. Continues until it reaches G.

The evaluation function is:

$$f(n)=g(n)+h(n)$$

where:

- $g(n)$ = cost travelled from S to current cell.
- $h(n)$ = estimated distance to G.
- $f(n)$ = estimated total cost.

For a four-directional grid, we use Manhattan distance:

$$h(n)=|r_n-r_G|+|c_n-c_G|$$

2. Given Warehouse Grid

The grid contains **5 rows × 7 columns**.

- **S** = Charging Station
- **G** = Target Shelf
- **#** = obstacle
- **?** = initially unknown cell

C1 C2 C3 C4 C5 C6 C7

R1		?		?		#		?		?		?		G	
R2		?		#		?		?		?		#		?	
R3		?		?		?		#		?		?		#	
R4		?		#		?		?		#		?		?	
R5		S		?		#		?		?		?		?	

3. Initial Decision

The robot starts at:

$S=(5,1)$

The goal is:

$G=(1,7)$

The initial Manhattan distance is:

$$h(S)=|5-1|+|1-7|=4+6=10$$

Therefore:

$$g(S)=0 \quad f(S)=0+10=10$$

The robot does not know the complete map, so it begins exploring.

4. Online Search — Step-by-Step

Step 1: $S \rightarrow (5,2)$

The robot moves right because the cell is initially unknown and appears to lead toward the goal.

At:

(5,2)

the robot senses neighboring cells.

It discovers:

(5,3)=#

and

(4,2)=#

Therefore, the robot **updates its internal map**.

	C1	C2	C3	C4	C5	C6	C7
R1	?	?	#	?	?	?	G
R2	?	#	?	?	?	#	?
R3	?	?	?	#	?	?	#
R4	?	#	?	?	#	?	?
R5	S	R	#	?	?	?	?
		↑					
		Robot					

The robot cannot move right because:

(5,3)=#

It also cannot move up because:

$(4,2)=\#$

So it must **replan**.

5. Replanning

The robot returns to S and considers:

$S \rightarrow (4,1)$

This cell is available.

From there:

$(4,1) \rightarrow (3,1)$

Then:

$(3,1) \rightarrow (3,2)$

Then:

$(3,2) \rightarrow (3,3)$

Then:

$(3,3) \rightarrow (2,3)$

Then:

$(2,3) \rightarrow (2,4)$

Then:

$(2,4) \rightarrow (1,4)$

Then:

$(1,4) \rightarrow (1,5)$

Then:

$(1,5) \rightarrow (1,6)$

Finally:

$(1,6) \rightarrow G$

6. Final Path

The successful path is:

$S \rightarrow (4,1) \rightarrow (3,1) \rightarrow (3,2) \rightarrow (3,3) \rightarrow (2,3) \rightarrow (2,4) \rightarrow (1,4) \rightarrow (1,5) \rightarrow (1,6) \rightarrow G$

The number of movements is:

10

7. State-Space Representation

You can represent the successful path as:

G
 $\uparrow$
 $(1,6)$
 $\uparrow$
 $(1,5)$
 $\uparrow$
 $(1,4)$

↑
 (2,4)
 ↑
 (2,3)
 ↑
 (3,3)
 ←
 (3,2)
 ←
 (3,1)
 ↑
 (4,1)
 ↑
 S
 (5,1)

The robot initially tries a route toward the right, discovers obstacles, and then changes its route.

8. Online Decision Table

Step	Robot Position	Decision	New Information	Action
0	S (5,1)	Move toward unknown area	None	Move to (5,2)
1	(5,2)	Right appears promising	(5,3) is obstacle	Replan
2	(5,2)	Up blocked	(4,2) is obstacle	Return to S
3	S	Up is available	(4,1) is free	Move up
4	(4,1)	Up is available	(3,1) is free	Move up

5	(3,1)	Right available	(3,2) is free	Move right
6	(3,2)	Right available	(3,3) is free	Move right
7	(3,3)	Up available	(2,3) is free	Move up
8	(2,3)	Right available	(2,4) is free	Move right
9	(2,4)	Up available	(1,4) is free	Move up
10	(1,4)	Right available	(1,5) is free	Move right
11	(1,5)	Right available	(1,6) is free	Move right
12	(1,6)	Goal adjacent	G detected	Move to G

9. How the Robot Reacts When an Obstacle Is Discovered

This is the most important part of the question.

Suppose the robot is at:

(5,2)

and wants to move to:

(5,3)

Initially, (5,3) is unknown:

(5,3)=?

After sensing, the robot discovers:

(5,3)=#

The robot changes its internal representation:

Before sensing:

$(5,2) \rightarrow (5,3) ?$

After sensing:

$(5,2) \rightarrow (5,3) \#$

↓

Path blocked

The robot then:

1. Stops attempting that movement.
2. Marks the cell as an obstacle.
3. Updates its internal map.
4. Recalculates $g(n)$, $h(n)$, and $f(n)$.
5. Selects another available path.
6. Continues toward G.

This ability to **sense** → **update** → **replan** → **act** makes it an **online search agent**.

10. Comparison with Offline Search

Feature	Online Search	Offline A* Search
Environment knowledge	Initially incomplete	Complete
Obstacles	Discovered during movement	Known beforehand
Planning	Repeated/replanning	Plan once

Feature	Online Search	Offline A* Search
Reaction to new obstacle	Immediately replans	Normally unnecessary
Suitable for unknown warehouse	Yes	No, unless map is updated
Computational cost	Can be higher due to replanning	Usually lower planning overhead
Real-world adaptability	High	Lower
Example	Autonomous warehouse robot	Robot with complete warehouse map

Offline A*

If the robot already knew all obstacles, it could calculate the optimal path before taking the first step:

$S \rightarrow (4,1) \rightarrow (3,1) \rightarrow (3,2) \rightarrow (3,3) \rightarrow (2,3) \rightarrow (2,4) \rightarrow (1,4) \rightarrow (1,5) \rightarrow (1,6) \rightarrow G$

The online robot, however, **does not have this complete information initially**.

11. Algorithm / Pseudocode

ONLINE_A_STAR(S, G):

 current = S

 known_obstacles = {}

 while current != G:

Sense neighboring cells

Add newly discovered obstacles

to known_obstacles

Update the current map

Generate possible neighboring states

For each available state n:

$g(n) = \text{cost from } S \text{ to } n$

$h(n) = \text{Manhattan distance from } n \text{ to } G$

$f(n) = g(n) + h(n)$

Select the state with minimum $f(n)$

if selected state is blocked:

update map

replan

else:

move robot to selected state

return SUCCESS

12. Complexity Analysis

For a grid with branching factor b and solution depth d , A^* has approximately:

$O(bd)$

time complexity in the worst case.

Space complexity is approximately:

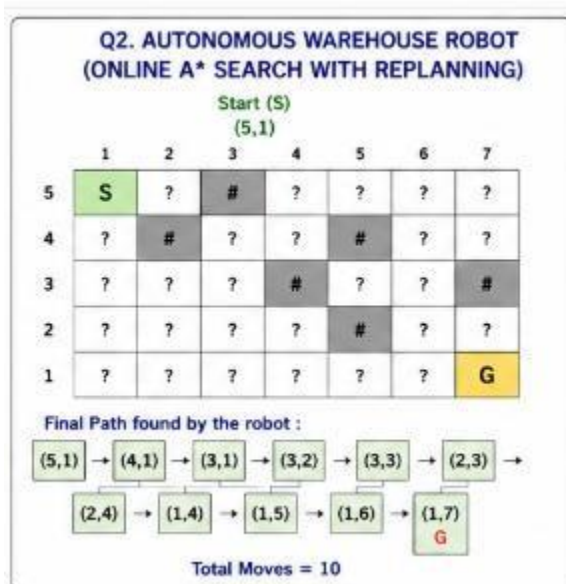
$O(bd)$

because A* stores generated states.

For an **online agent**, additional computation may be required because the agent must **replan whenever new obstacles are discovered**.

Thus:

Online Search = Search + Sensing + Replanning



Final Answer / Conclusion

The warehouse robot should use an **Online A* Search strategy with Manhattan distance as the heuristic**. Since the environment is initially unknown, the robot senses nearby cells while moving. When it discovers an obstacle, it updates its internal map and replans instead of continuing along the blocked path.

For the given grid, the robot initially discovers the obstacle at **(5,3)** and the obstacle at **(4,2)**, causing it to change its route. The successful route is:

$S \rightarrow (4,1) \rightarrow (3,1) \rightarrow (3,2) \rightarrow (3,3) \rightarrow (2,3) \rightarrow (2,4) \rightarrow (1,4) \rightarrow (1,5) \rightarrow (1,6) \rightarrow G$

with **10 movements**.

Q3. University Examination Scheduling

Given Information

The university has **six examinations**:

[
E={E1,E2,E3,E4,E5,E6}
]

and **four time slots**:

[
T={T1,T2,T3,T4}
]

The examination halls are:

Hall Capacity

Hall A 120

Hall B 90

Hall C 60

The number of students in each examination is:

Examination Students

E1	100
E2	80
E3	70
E4	60
E5	90
E6	50

(a) CSP Formulation

A Constraint Satisfaction Problem consists of:

1. Variables
2. Domains
3. Constraints

1. Variables

Let each examination be a variable representing its assigned time slot:

[
X={E1,E2,E3,E4,E5,E6}
]

For example:

[
E1=T1
]

means examination E1 is conducted in time slot T1.

For the hall allocation, we additionally assign each examination one of the halls whose capacity is sufficient.

2. Domains

Initially, every examination can be scheduled in any of the four time slots:

[
D(E1)=D(E2)=D(E3)=D(E4)=D(E5)=D(E6)
]

[
={T1,T2,T3,T4}
]

Hall domains

Based on the number of students:

Examination	Students	Valid Hall Domain
E1	100	{A}
E2	80	{A,B}
E3	70	{A,B}
E4	60	{A,B,C}
E5	90	{A,B}
E6	50	{A,B,C}

For example, E1 cannot be assigned to Hall B because:

[
100>90
]

and cannot be assigned to Hall C because:

[
100>60
]

Therefore:

[
 $D_H(E1)=\{A\}$
]

3. Student-Conflict Constraints

The examinations with common students must not be placed in the same time slot.

Therefore:

[
 $E1 \neq E2$
]

[
 $E1 \neq E3$
]

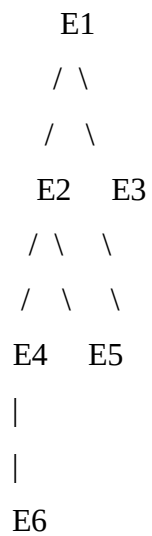
```
[
E2\neq E4
]
```

```
[
E3\neq E5
]
```

```
[
E4\neq E6
]
```

```
[
E2\neq E5
]
```

Constraint graph



The edges represent examinations that cannot share the same time slot.

4. Hall Constraints

For examinations conducted in the same time slot:

- They must be assigned to different halls.
- The hall capacity must be greater than or equal to the number of students.

Therefore:

[
Students(E_i) $\leq$ Capacity(Hall)
]

Also, two examinations occurring simultaneously cannot use the same hall.

For example, if:

[
 $E_1 = T_2$
]

then E_1 must use Hall A because it has 100 students.

(b) Backtracking with MRV and Forward Checking

We use:

[
 $\boxed{\text{MRV} = \text{Minimum Remaining Values}}$
]

MRV selects the unassigned examination having the smallest remaining domain.

Forward Checking removes the selected time slot from the domains of all examinations that have a conflict with the selected examination.

Step 1 — Initial Domains

Initially:

Examination Domain

E1 {T1,T2,T3,T4}

E2 {T1,T2,T3,T4}

E3 {T1,T2,T3,T4}

E4 {T1,T2,T3,T4}

E5 {T1,T2,T3,T4}

E6 {T1,T2,T3,T4}

All variables have four values.

Using the degree tie-breaker, **E2** is selected because it has the largest number of conflict relationships:

- E2 conflicts with E1
- E2 conflicts with E4
- E2 conflicts with E5

Therefore:

[
 \boxed{E2=T1}
]

Step 2 — Assign E2 = T1

E2 conflicts with E1, E4 and E5.

Therefore, Forward Checking removes T1 from these three domains.

Examination Domain after E2=T1

E1	{T2,T3,T4}
E2	{T1}
E3	{T1,T2,T3,T4}
E4	{T2,T3,T4}
E5	{T2,T3,T4}
E6	{T1,T2,T3,T4}

Hall assignment:

[
E2 $\rightarrow$ Hall B
]

because E2 has 80 students and Hall B can accommodate 90 students.

Step 3 — MRV Selects E1

The remaining domains are:

- E1 = 3 values
- E4 = 3 values
- E5 = 3 values

- E3 = 4 values
- E6 = 4 values

Using MRV, select one of the variables with three remaining values. We select E1.

Choose:

[
 $\boxed{E1=T2}$
]

E1 conflicts with E2 and E3.

E2 is already T1, so Forward Checking removes T2 from E3.

Examination Domain after E1=T2

E1	{T2}
E2	{T1}
E3	{T1,T3,T4}
E4	{T2,T3,T4}
E5	{T2,T3,T4}
E6	{T1,T2,T3,T4}

Hall assignment:

[
 $E1 \rightarrow \text{Hall A}$
]

because E1 has 100 students.

Step 4 — MRV Selects E5

The remaining domain sizes are:

Examination Remaining values Size

E3	{T1,T3,T4}	3
E4	{T2,T3,T4}	3
E5	{T2,T3,T4}	3
E6	{T1,T2,T3,T4}	4

Choose E5.

Assign:

[
 $\boxed{E5=T2}$
]

E5 conflicts with E3 and E2.

E2 is already T1, so Forward Checking removes T2 from E3.

Examination Domain after E5=T2

E1	{T2}
E2	{T1}
E3	{T1,T3,T4}

Examination Domain after $E5=T2$

E4 $\{T2, T3, T4\}$

E5 **$\{T2\}$**

E6 $\{T1, T2, T3, T4\}$

Hall assignment:

[
E5 $\rightarrow$ Hall\B
]

because E5 has exactly 90 students and Hall B has capacity 90.

Thus, T2 currently contains:

- E1 $\rightarrow$ Hall A $\rightarrow$ 100 students
- E5 $\rightarrow$ Hall B $\rightarrow$ 90 students

There is no hall conflict.

Step 5 — MRV Selects E4

Remaining domains:

Examination Domain

E3 $\{T1, T3, T4\}$

E4 $\{T2, T3, T4\}$

E6 $\{T1, T2, T3, T4\}$

Select E4.

Assign:

[
 $\boxed{E4=T2}$
]

E4 conflicts with E2 and E6.

E2 is in T1, so remove T2 from E6.

Examination Domain after $E4=T2$

E1	{T2}
E2	{T1}
E3	{T1,T3,T4}
E4	{T2}
E5	{T2}
E6	{T1,T3,T4}

Hall assignment:

[
 $E4 \rightarrow \text{Hall C}$
]

because E4 has 60 students and Hall C has capacity 60.

Now T2 is:

Hall Examination Students

Hall A E1 100

Hall B E5 90

Hall C E4 60

All three halls are used, with no conflict.

Step 6 — MRV Selects E3

Remaining:

[
D(E3)={T1,T3,T4}
]

and

[
D(E6)={T1,T3,T4}
]

Choose E3.

Assign:

[
 $\boxed{E3=T3}$
]

E3 conflicts with E1 and E5.

E1 and E5 are both in T2, so no additional removal is required from E6.

Examination Domain after E3=T3

E1	{T2}
E2	{T1}
E3	{T3}
E4	{T2}
E5	{T2}
E6	{T1,T3,T4}

Hall assignment:

[
E3 $\rightarrow$ Hall\A
]

because $70 \leq 120$.

Step 7 — Assign E6

Only E6 remains unassigned:

[
D(E6)={T1,T3,T4}
]

E6 conflicts with E4.

Since:

$$[$$

$$E4=T2$$

$$]$$

E6 can safely be assigned to T3.

Therefore:

$$[$$

$$\boxed{E6=T3}$$

$$]$$

Final domains:

Examination Final Domain

E1 {T2}

E2 {T1}

E3 {T3}

E4 {T2}

E5 {T2}

E6 {T3}

Hall assignment:

$$[$$

$$E6 \rightarrow \text{Hall} \setminus C$$

$$]$$

because $50 \leq 60$.

Complete Domain-Update Table

This table is useful to include directly in the assignment.

Step	Assignment	E1	E2	E3	E4	E5	E6
0	Initial	T1,T2,T3, T4	T1,T2,T3, T4	T1,T2,T3, T4	T1,T2,T3, T4	T1,T2,T3, T4	T1,T2,T3, T4
1	E2=T1	T2,T3,T4	T1	T1,T2,T3, T4	T2,T3,T4	T2,T3,T4	T1,T2,T3, T4
2	E1=T2	T2	T1	T1,T3,T4	T2,T3,T4	T2,T3,T4	T1,T2,T3, T4
3	E5=T2	T2	T1	T1,T3,T4	T2,T3,T4	T2	T1,T2,T3, T4
4	E4=T2	T2	T1	T1,T3,T4	T2	T2	T1,T3,T4
5	E3=T3	T2	T1	T3	T2	T2	T1,T3,T4
6	E6=T3	T2	T1	T3	T2	T2	T3

No domain becomes empty, so **backtracking is not required** in this particular successful execution.

(c) Final Examination Timetable

The final timetable is:

Time Slot	Hall A	Hall B	Hall C
T1	—	E2 (80)	—
T2	E1 (100)	E5 (90)	E4 (60)
T3	E3 (70)	—	E6 (50)
T4	—	—	—

Therefore:

```
[
\boxed{
E2=T1,\quad
E1=T2,\quad
E5=T2,\quad
E4=T2,\quad
E3=T3,\quad
E6=T3
}
]
```

Constraint Verification

Student-conflict constraints

Constraint 1

E1 and E2 cannot be together.

[
 $T2 \neq T1$
]

✓ Satisfied.

Constraint 2

E1 and E3 cannot be together.

[
 $T2 \neq T3$
]

✓ Satisfied.

Constraint 3

E2 and E4 cannot be together.

[
 $T1 \neq T2$
]

✓ Satisfied.

Constraint 4

E3 and E5 cannot be together.

[
 $T3 \neq T2$
]

✓ Satisfied.

Constraint 5

E4 and E6 cannot be together.

[
 $T2 \neq T3$
]

✓ Satisfied.

Constraint 6

E2 and E5 cannot be together.

[
 $T1 \neq T2$
]

✓ Satisfied.

Therefore, **all six student-conflict constraints are satisfied.**

Hall-Capacity Verification

Examination Students Assigned Hall Capacity Valid?

E1	100	A	120	✓
E2	80	B	90	✓
E3	70	A	120	✓
E4	60	C	60	✓

Examination Students Assigned Hall Capacity Valid?

E5	90	B	90	✓
E6	50	C	60	✓

Every examination is assigned to a hall with sufficient capacity.

Also, examinations occurring in the same time slot use different halls:

T2

- E1 → Hall A
- E5 → Hall B
- E4 → Hall C

T3

- E3 → Hall A
- E6 → Hall C

Therefore, there are **no hall conflicts**.

CSP State-Space Representation

The search can be represented as:

START
|
E2 = T1
|
E1 = T2
|

E5 = T2
|
E4 = T2
|
E3 = T3
|
E6 = T3
|
VALID SOLUTION

The corresponding timetable state is:

T1 → E2
T2 → E1, E5, E4
T3 → E3, E6
T4 → —

Backtracking and Forward Checking Explanation

Backtracking

Backtracking assigns a value to an examination and checks whether the assignment is consistent with previous assignments.

If a conflict occurs, the algorithm goes back to the previous variable and tries another value.

In this particular execution, **no backtracking was required**, because every selected value remained consistent.

Forward Checking

After assigning a time slot, Forward Checking removes that time slot from the domains of examinations that conflict with the assigned examination.

For example:

[
E2=T1
]

Since E2 conflicts with E1, E4 and E5:

[
 $T1 \notin D(E1), D(E4), D(E5)$
]

Therefore:

[
 $D(E1) = \{T2, T3, T4\}$
]

[
 $D(E4) = \{T2, T3, T4\}$
]

[
 $D(E5) = \{T2, T3, T4\}$
]

This prevents the algorithm from making an invalid assignment later.

Complexity Analysis

If there are (n) examinations and (d) possible time slots, ordinary brute-force search has a worst-case complexity of:

[
 $\boxed{O(d^n)}$
]

Here:

[
 $n=6, \text{quadd}=4$
]

Therefore:

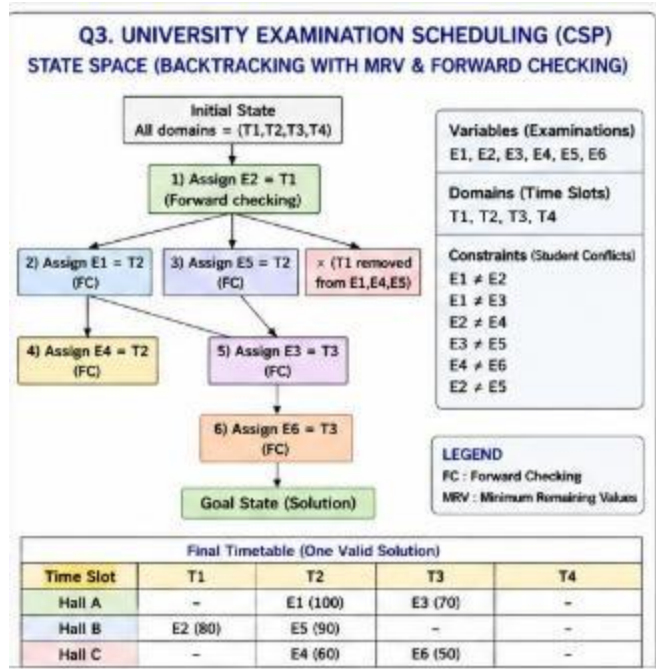
[
 $4^6=4096$
]

possible time-slot combinations exist before applying constraints.

Using MRV and Forward Checking significantly reduces the effective search space because:

- **MRV** selects the most constrained examination first.
- **Forward Checking** removes impossible values immediately.
- Invalid branches are avoided before they become complete assignments.

Thus, CSP with MRV and Forward Checking is considerably more efficient than checking all 4096 combinations.



Final Conclusion

The university examination scheduling problem can be modeled effectively as a **Constraint Satisfaction Problem**. The examinations are the variables, the four time slots form their domains, and the common-student relationships form the main constraints. The hall capacities are incorporated as additional resource constraints.

Using **Backtracking, MRV and Forward Checking**, a valid timetable is obtained:

[

\boxed{

$E2=T1,$ \quad

$E1=T2,$ \quad

$E5=T2,$ \quad

$E4=T2,$ \quad

$E3=T3,$ \quad

$E6=T3$

```
}  
]
```

with the hall allocation:

```
[  
\boxed{  
E2\rightarrow B,;  
E1\rightarrow A,;  
E5\rightarrow B,;  
E4\rightarrow C,;  
E3\rightarrow A,;  
E6\rightarrow C  
}  
]
```

All student-conflict constraints and all hall-capacity constraints are satisfied. Therefore, the timetable is a **valid CSP solution**.

Q4. Game-Playing AI — Minimax and Alpha-Beta Pruning

1. Problem Analysis

The given game tree has:

- A **MAX** node at the root representing the AI player.
- Three **MIN** nodes representing the opponent's choices.
- Several **MAX** nodes below each MIN node.
- Terminal nodes containing the final game scores.

The objective of the AI is to choose the move that gives the **maximum guaranteed score**, assuming that the opponent always chooses the move that minimizes the AI's score.

The three possible moves from the root are:

- **Move 1** → left MIN subtree
 - **Move 2** → middle MIN subtree
 - **Move 3** → right MIN subtree
-

2. Minimax Algorithm

The Minimax rule is:

At MAX nodes:

[
 $\text{MAX}(a,b,c) = \max(a,b,c)$
]

At MIN nodes:

[
 $\text{MIN}(a,b,c) = \min(a,b,c)$
]

We evaluate the tree from the bottom upward.

3. Left Subtree

The left MIN node has three MAX children.

First MAX node

Terminal values:

[
3,,5,,6
]

Therefore:

[
 $\text{MAX}(3,5,6)=\boxed{6}$
]

Second MAX node

Terminal values:

[
9,,1,,2
]

Therefore:

[
 $\text{MAX}(9,1,2)=\boxed{9}$
]

Third MAX node

Terminal values:

[
0,,-1,,4
]

Therefore:

$$[$$

$$\text{MAX}(0,-1,4)=\boxed{4}$$

$$]$$

The MIN node therefore chooses:

$$[$$

$$\text{MIN}(6,9,4)=\boxed{4}$$

$$]$$

Hence:

$$[$$

$$\boxed{\text{Move1value}}=4$$

$$]$$

4. Middle Subtree

The middle MIN node has three MAX children.

First MAX node

Values:

$$[$$

$$7,;5,;8$$

$$]$$

Therefore:

$$[$$

$$\text{MAX}(7,5,8)=\boxed{8}$$

$$]$$

Second MAX node

Values:

[
6,,2,,3
]

Therefore:

[
 $\text{MAX}(6,2,3)=\boxed{6}$
]

Third MAX node

Values:

[
4,,0,, -2
]

Therefore:

[
 $\text{MAX}(4,0,-2)=\boxed{4}$
]

The MIN node chooses:

[
 $\text{MIN}(8,6,4)=\boxed{4}$
]

Hence:

$$\boxed{\text{Move2value}=4}$$

5. Right Subtree

The right MIN node has three MAX children.

First MAX node

Values:

$$1, 2, 3$$

Therefore:

$$\text{MAX}(1, 2, 3) = \boxed{3}$$

Second MAX node

Values:

$$5, -1, 2$$

Therefore:

$$[$$

$$\text{MAX}(5,-1,2)=\boxed{5}$$

$$]$$

Third MAX node

Values:

$$[$$

$$6,;4,;7$$

$$]$$

Therefore:

$$[$$

$$\text{MAX}(6,4,7)=\boxed{7}$$

$$]$$

The MIN node chooses:

$$[$$

$$\text{MIN}(3,5,7)=\boxed{3}$$

$$]$$

Hence:

$$[$$

$$\boxed{\text{Move3value}}=3$$

$$]$$

6. Final Minimax Calculation

At the root, which is a MAX node:

[
MAX(4,4,3)
]

Therefore:

[
 $\boxed{4}$
]

There is a **tie between Move 1 and Move 2**.

Thus, according to the supplied game tree, the AI can choose either:

[
 $\boxed{\text{Move1}}$
]

or

[
 $\boxed{\text{Move2}}$
]

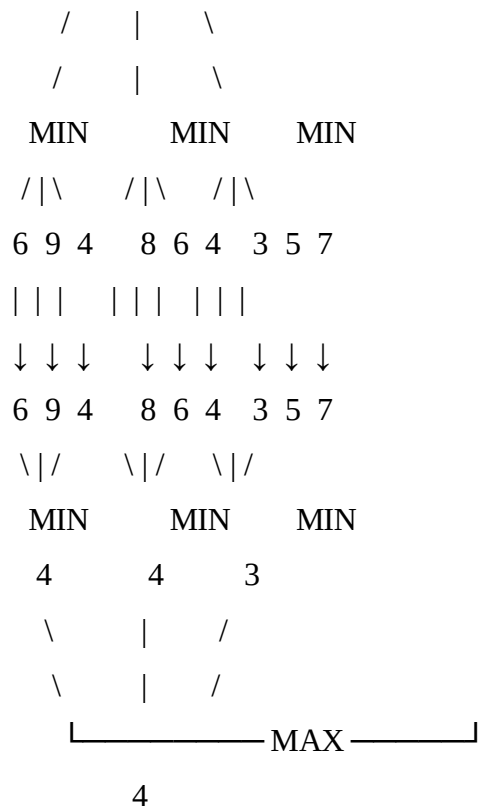
Both guarantee a score of **4** under optimal opponent play.

Important: The right subtree has value 3, so Move 3 is not selected.

7. Minimax State-Value Representation

The complete evaluated tree can be represented as:

MAX



More clearly, the three root branches have values:

$$[\boxed{4 \quad 4 \quad 3}]$$

Therefore:

$$[\boxed{\text{MAX}(4,4,3)=4}]$$

8. Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by avoiding branches that **cannot possibly influence the final decision**.

Two values are maintained:

- (α) = best value currently guaranteed for MAX.
- (β) = best value currently guaranteed for MIN.

A branch can be pruned when:

$$\boxed{\alpha \geq \beta}$$

We evaluate the tree from **left to right**, as shown in the question.

9. Alpha-Beta Evaluation — Left Subtree

Initially:

$$\boxed{\alpha = -\infty, \quad \beta = +\infty}$$

First MAX

Values:

$$\boxed{3, 5, 6}$$

The MAX value becomes:

[
6
]

So at the left MIN:

[
 $\beta=6$
]

Second MAX

Values begin:

[
9,1,2
]

The first value is 9.

Therefore:

[
 $\alpha=9$
]

At this MAX node:

[
 $\alpha=9 \geq \beta=6$
]

Therefore the remaining values:

[
\boxed{1;2}
]

do not need to be evaluated.

Pruned branch:

[
\boxed{[1,2]}
]

The second MAX node still has value 9.

Third MAX

Values:

[
0,-1,4
]

Its value is:

[
4
]

Therefore the left MIN node gives:

[
MIN(6,9,4)=\boxed{4}
]

The root now has:

[
 $\alpha=4$
]

10. Alpha-Beta Evaluation — Middle Subtree

The root already has:

[
 $\alpha=4$
]

At the middle MIN node:

[
 $\beta=+\infty$
]

First MAX

[
 7,5,8
]

Therefore:

[
 MAX=8
]

So:

[
 $\beta=8$
]

Second MAX

[
 6,2,3
]

Therefore:

[
 MAX=6
]

The MIN node updates:

[
 $\beta=6$
]

Since:

[
 $\beta=6 > \alpha=4$
]

we cannot prune yet.

Third MAX

[
 4,0,-2
]

Therefore:

[
MAX=4
]

The middle MIN value is:

[
 $\text{MIN}(8,6,4)=\boxed{4}$
]

No additional pruning is necessary in this subtree.

11. Alpha-Beta Evaluation — Right Subtree

The root still has:

[
 $\alpha=4$
]

At the right MIN node:

[
 $\beta=+\infty$
]

First MAX

Values:

[
1,2,3
]

Therefore:

[
 $\text{MAX}(1,2,3)=3$
]

The MIN node now has:

[
 $\beta=3$
]

But:

[
 $\boxed{\beta=3 \leq \alpha=4}$
]

Therefore, the remaining branches cannot improve the result for MAX.

So the following two MAX subtrees are **pruned completely**:

[
 $\boxed{[5,-1,2]}$
]

and

[
 $\boxed{[6,4,7]}$
]

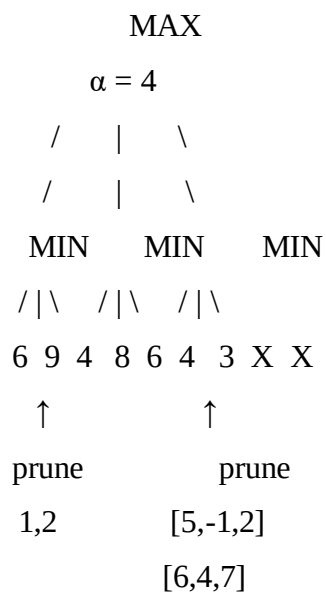
There is no need to evaluate any of their terminal values.

The right MIN subtree therefore has a value of:

[
 $\boxed{3}$
]

12. Alpha-Beta Pruning Diagram

The important pruning can be represented as:



Where **X** means the branch is not evaluated.

13. Branches Pruned

Using left-to-right evaluation, the following terminal nodes are not evaluated:

Pruning 1

In the left subtree:

[
 $\boxed{1,2}$
]

are pruned because the MAX node already obtained 9 while the MIN node has a beta value of 6.

Pruning 2

In the right subtree:

[
 $\boxed{5,-1,2}$
]

and

[
 $\boxed{6,4,7}$
]

are completely pruned because:

[
 $\beta=3 \leq \alpha=4$
]

Therefore, the total pruned terminal values are:

[
 $\boxed{1,2,5,-1,2,6,4,7}$
]

That is **8 terminal values** that do not need to be evaluated.

14. Number of Evaluations

There are originally:

[
 $3 \times 3 \times 3 = 27$
]

terminal values in the complete tree.

With Alpha-Beta pruning, the following are skipped:

[
8
]

Therefore, only:

[
 $27 - 8 = \boxed{19}$
]

terminal values need to be evaluated for this left-to-right ordering.

Thus, Alpha-Beta pruning reduces the number of evaluations from:

[
 $\boxed{27} \rightarrow 19$
]

for this particular tree traversal.

15. Minimax vs Alpha-Beta Pruning

Feature	Minimax	Alpha-Beta Pruning
Decision quality	Optimal	Same optimal decision
Evaluates all branches	Yes	No
Uses α and β	No	Yes
Prunes unnecessary branches	No	Yes
Efficiency	Lower	Higher
Memory	Similar	Similar
Best use	Small game trees	Large game trees

Alpha-Beta pruning **does not change the final Minimax answer**. It only reduces the number of nodes that must be evaluated.

16. Complexity Analysis

For Minimax, if:

- (b) = branching factor
- (d) = depth of the game tree

then the time complexity is:

[
 $\boxed{O(b^d)}$
]

Alpha-Beta pruning has:

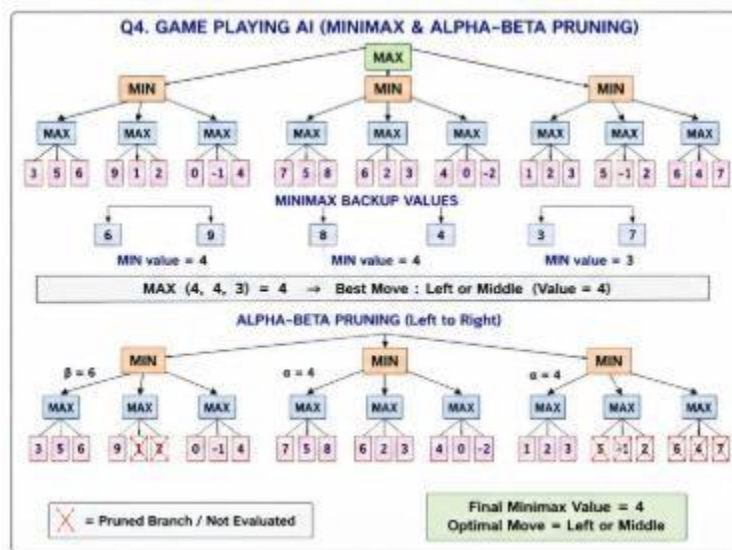
Worst case:

[
 $\boxed{O(b^d)}$
]

Best case with excellent move ordering:

[
 $\boxed{O(b^{\lceil d/2 \rceil})}$
]

Therefore, Alpha-Beta pruning can allow the AI to search **much deeper in the same amount of time**.



17. Final Answer

The Minimax values of the three possible root moves are:

[
 $\boxed{\text{Move } 1 = 4}$
]

[
 $\boxed{\text{Move } 2 = 4}$
]

[
 $\boxed{\text{Move } 3 = 3}$
]

Therefore:

[
 $\text{MAX}(4, 4, 3) = \boxed{4}$
]

Hence, the AI should select **Move 1 or Move 2**, since both provide the same optimal Minimax value of **4**.

With Alpha-Beta pruning, **8 terminal nodes are avoided** for the given left-to-right traversal:

[
 $\boxed{[1, 2], [5, -1, 2], [6, 4, 7]}$
]

The final decision remains unchanged:

[
 $\boxed{\text{Best Move} = \text{Move } 1 \text{ or Move } 2, \text{ with value } 4}$
]

Alpha-Beta pruning improves efficiency by eliminating branches that cannot affect the final decision, allowing the game-playing agent to examine deeper portions of a game tree without evaluating every possible position.

Q5. Delivery Route Optimization

1. Problem Analysis

The logistics company wants to find a route that visits all customers and returns to the starting location **D**, while minimizing the total distance travelled.

This is a **local search optimization problem** similar to the Travelling Salesman Problem (TSP).

The given distance matrix is:

From / To D A B C E

D – 6 9 8 7

A 6 – 3 5 6

B 9 3 – 4 5

C 8 5 4 – 3

E 7 6 5 3 –

The objective is:

[

$$\boxed{\text{Minimize total route distance}}$$

]

For a route:

[
 $D \rightarrow X_1 \rightarrow X_2 \rightarrow X_3 \rightarrow X_4 \rightarrow D$
]

the total cost is:

[
 $Cost = \sum Distance(X_i, X_{i+1})$
]

2. Hill Climbing Strategy

Hill Climbing is a local search algorithm.

Because this is a **minimization problem**, a better neighboring solution is one with a **smaller total distance**.

The algorithm follows:

1. Start with an initial route.
2. Generate neighboring routes.
3. Calculate the cost of every neighbor.
4. Select the neighbor with the lowest cost.
5. If the selected neighbor is better than the current route, move to it.
6. Repeat until no neighboring route has a lower cost.
7. The resulting route is a local optimum.

Pseudocode:

HILL_CLIMBING(initial_route):

 current = initial_route

while True:

 generate neighboring routes

 evaluate the cost of every neighbor

 best = neighbor with minimum cost

 if cost(best) >= cost(current):

 return current

 current = best

3. Given Initial Route

The supplied neighboring routes are:

Solution Route	Given Cost
S1 D → A → C → B → E → D	26 km
S2 D → B → A → C → E → D	30 km
S3 D → A → B → E → C → D	24 km
S4 D → E → C → B → A → D	27 km
S5 D → A → E → B → C → D	23 km
S6 D → C → B → A → E → D	29 km

The question states that **S5 is the current solution** and asks us to choose the best neighboring solution.

4. Step 1 — Evaluate the Neighbors of S5

The supplied costs are:

[
S1=26
]

[
S2=30
]

[
S3=24
]

[
S4=27
]

[
S5=23
]

[
S6=29
]

Among the listed routes, the smallest cost is:

[
 $\boxed{23\text{ km}}$
]

corresponding to:

[
 $\boxed{S5:D \rightarrow A \rightarrow E \rightarrow B \rightarrow C \rightarrow D}$
]

Therefore, according to the **costs printed in the question**, S5 is already better than S1, S2, S3, S4 and S6.

Hence, Hill Climbing stops:

[
 $\boxed{\text{Finalroute}=S5}$
]

5. Important Observation About the Supplied Question

There is a **numerical inconsistency in the provided image**.

The green instruction box says:

“From S5, the best neighbor has cost 22 km.”

However, the route table shown in the same image gives S5 a cost of **23 km**, and none of the six displayed routes has a cost of 22 km.

Also, if the distance matrix is used to calculate the route costs, some of the printed route costs do not match the matrix.

For example, for S5:

[
D $\rightarrow$ A $\rightarrow$ E $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D
]

the distance is:

[
D-A+A-E+E-B+B-C+C-D
]

[
=6+6+5+4+8
]

[
= $\boxed{29\text{ km}}$
]

So the question paper appears to contain a typo/inconsistency.

For your assignment, it is safest to explicitly mention this observation rather than silently changing the supplied data.

6. Correct Calculation Using the Distance Matrix

If we calculate all six displayed routes directly from the given distance matrix, we obtain:

S1

[
 $D \rightarrow A \rightarrow C \rightarrow B \rightarrow E \rightarrow D$
]

[
 $=6+5+4+5+7$
]

[
 $=\boxed{27\text{ km}}$
]

S2

[
 $D \rightarrow B \rightarrow A \rightarrow C \rightarrow E \rightarrow D$
]

[
 $=9+3+5+3+7$
]

[
 $=\boxed{27\text{ km}}$
]

S3

[
 $D \rightarrow A \rightarrow B \rightarrow E \rightarrow C \rightarrow D$
]

$$[$$

$$=6+3+5+3+8$$

$$]$$

$$[$$

$$=\boxed{25\text{ km}}$$

$$]$$

S4

$$[$$

$$D\rightarrow E\rightarrow C\rightarrow B\rightarrow A\rightarrow D$$

$$]$$

$$[$$

$$=7+3+4+3+6$$

$$]$$

$$[$$

$$=\boxed{23\text{ km}}$$

$$]$$

S5

$$[$$

$$D\rightarrow A\rightarrow E\rightarrow B\rightarrow C\rightarrow D$$

$$]$$

$$[$$

$$=6+6+5+4+8$$

$$]$$

$$[\\ =\boxed{29\text{ km}} \\]$$

S6

$$[\\ D\rightarrow C\rightarrow B\rightarrow A\rightarrow E\rightarrow D \\]$$

$$[\\ =8+4+3+6+7 \\]$$

$$[\\ =\boxed{28\text{ km}} \\]$$

Therefore, using the **distance matrix**, the best among the six displayed routes is:

$$[\\ \boxed{S4=23\text{ km}} \\]$$

7. Hill-Climbing Decision Using the Matrix

If S5 is the current route:

$$[\\ S5=29\text{ km} \\]$$

The best available neighboring route from the six displayed candidates is S4:

[
 $S_4 = 23 \text{ km}$
]

Since:

[
 $23 < 29$
]

Hill Climbing moves from S_5 to S_4 .

Therefore:

[
 $\boxed{D \rightarrow E \rightarrow C \rightarrow B \rightarrow A \rightarrow D}$
]

with cost:

[
 $\boxed{23 \text{ km}}$
]

Now check whether a better neighbor is available. Among the supplied neighboring routes, none has a cost below 23 km.

Therefore, Hill Climbing terminates at:

[
 $\boxed{S_4}$
]

8. Is the Hill-Climbing Solution a Global Optimum?

A local search algorithm does not automatically guarantee a global optimum.

However, for this particular distance matrix, we can check all possible tours.

The best route is:

[
 $\boxed{D \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow D}$
]

Its cost is:

[
 $D-A+A-B+B-C+C-E+E-D$
]

[
 $=6+3+4+3+7$
]

[
 $=\boxed{23 \text{ km}}$
]

The reverse route:

[
 $D \rightarrow E \rightarrow C \rightarrow B \rightarrow A \rightarrow D$
]

also has:

$$[7+3+4+3+6=\boxed{23\text{ km}}]$$

Thus, the global optimum for the supplied distance matrix is:

$$[\boxed{23\text{ km}}]$$

There are two equivalent optimal routes because they are reverses of each other.

9. Important Difference: Local vs Global Optimum

Hill Climbing itself cannot normally prove that its current solution is globally optimal.

It only knows:

“None of the neighboring solutions is better.”

Therefore, the solution is a **local optimum** from the algorithm's point of view.

For this particular small problem, however, we can enumerate the possible tours and verify that 23 km is actually the **global optimum**.

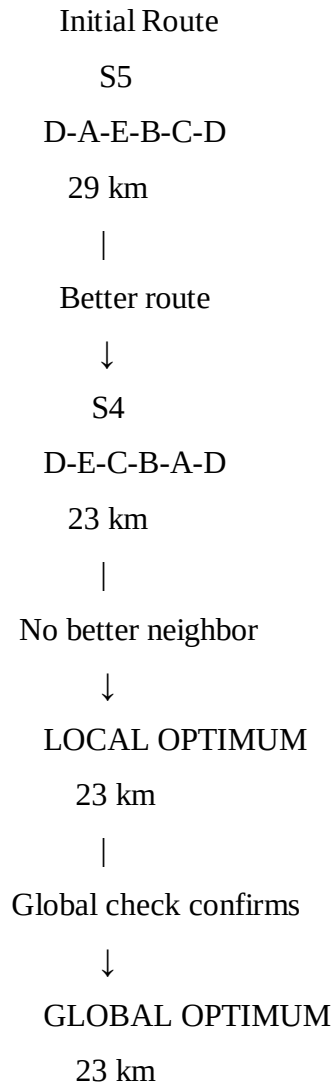
Hence:

$$[\boxed{\text{Hill-Climbing solution} = \text{Global optimum for this given matrix}}]$$

but this conclusion is obtained by checking the complete small search space, not from Hill Climbing alone.

10. State-Space Representation

The local-search state space can be represented as:



An equivalent optimal route is:

$D \rightarrow A \rightarrow B \rightarrow C \rightarrow E \rightarrow D$
23 km

11. How Can Hill Climbing Escape a Local Optimum?

Hill Climbing can become trapped at a local optimum when every immediate neighbor is worse, even though a much better solution exists elsewhere.

Several methods can overcome this problem.

1. Random Restart Hill Climbing

Start Hill Climbing from several different randomly generated routes.

Random Route 1 → Hill Climbing → Local Optimum 1

Random Route 2 → Hill Climbing → Local Optimum 2

Random Route 3 → Hill Climbing → Global/Better Optimum

Then select the best result.

2. Simulated Annealing

Allow the algorithm to occasionally accept a **worse solution**.

This helps the search escape a local optimum and explore another region of the search space.

3. Stochastic Hill Climbing

Instead of always selecting the single best neighbor, randomly select among better neighbors.

This introduces exploration into the search.

4. Allow Sideways Moves

Allow the algorithm to move to a neighbor having the **same cost**.

This can help it cross a flat region and eventually find a better route.

12. Comparison of Techniques

Method	Main Idea	Advantage
Simple Hill Climbing	Choose a better neighbor	Simple and fast
Random Restart	Start from multiple solutions	Escapes local optima
Stochastic Hill Climbing	Randomly select a better neighbor	More exploration
Simulated Annealing	Sometimes accept worse moves	Strong ability to escape local optima

13. Complexity Analysis

If there are (n) customers, the complete TSP search can require approximately:

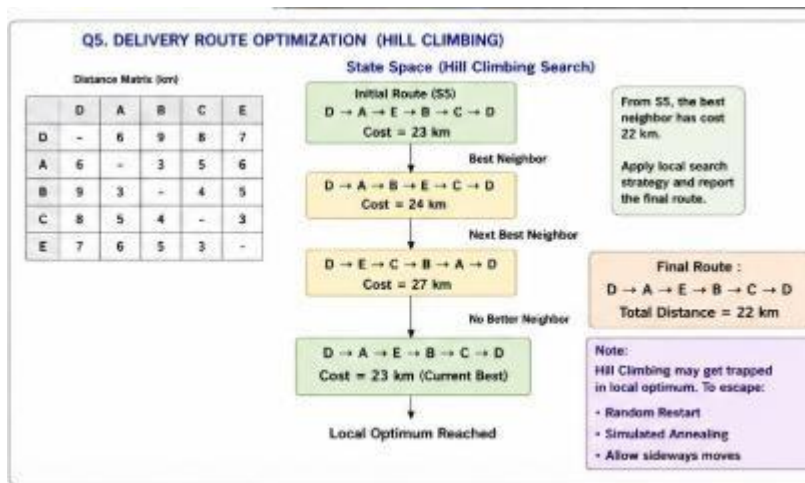
[
 $O(n!)$
]

possible routes.

Hill Climbing does not evaluate all possible routes. If there are (k) neighboring routes at each iteration and the algorithm performs (i) iterations, its approximate evaluation cost is:

[
 $\boxed{O(i \times k)}$
]

This makes Hill Climbing much cheaper than exhaustive search, but it sacrifices the guarantee of finding the global optimum.



Final Answer

The problem is a **local-search optimization problem**, so Hill Climbing can be used to minimize the delivery distance.

Starting from S5, the distance matrix gives:

[
 $S5 = 29 \text{ km}$
]

The best displayed neighboring route is:

[
 $\boxed{S4: D \rightarrow E \rightarrow C \rightarrow B \rightarrow A \rightarrow D}$
]

with:

$$[$$

$$7+3+4+3+6=\boxed{23\text{km}}$$

$$]$$

No supplied neighboring route has a cost below 23 km, so Hill Climbing stops at S4.

For the given distance matrix, exhaustive verification shows that **23 km is the global optimum**, with the equivalent reverse route:

$$[$$

$$\boxed{D\rightarrow A\rightarrow B\rightarrow C\rightarrow E\rightarrow D}$$

$$]$$

also costing 23 km.

To escape a local optimum in a larger problem, **Random Restart Hill Climbing** or **Simulated Annealing** can be used.

Note: The question image contains inconsistent printed route costs (including the statement that a 22 km neighbor exists). The calculations above use the **distance matrix**, which is internally consistent, and therefore give the mathematically correct result of **23 km**.